



hellokitty2

随笔 - 684, 文章 - 1, 评论 - 21, 阅读 - 102万

导航

博客园
首页
新随笔
联系
订阅 
管理

< 2025年5月 >						
日	一	二	三	四	五	六
27	28	29	30	1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31
1	2	3	4	5	6	7

公告

昵称： Hello-World3
园龄： 8年2个月
粉丝： 113
关注： 7
[+加关注](#)

搜索

pinctrl(1)——pinctrl子系统的使用

一、pinctrl子系统设备树配置

有了pinctrl子系统以后，驱动就可以操作pinctrl子系统的接口函数完成I/O操作了，而不需要自己去配置了。一般pinctrl子系统驱动是由芯片原厂的BSP工程师实现好的。驱动工程师通过配置设备树去使用pinctrl子系统。有些I/O口具有不同的状态(state)，比如在正常工作的时候这一组I/O口被配置成uart接口，休眠时配置成GPIO接口且输出为高电平。pinctrl子系统的设备树配置也是遵守service和client结构。

举个例子：这里的device节点成为pinctrl子系统中的一个client设备，因为其使用了pinctrl子系统里面提供出来的接口。pinctrl就是pincontroller的缩写。



```
//client节点
device {
    pinctrl-names = "default", "sleep"; //使用pinctrl-names来表示设备的状态(state),这里1
    pinctrl-0 = <&state_0_node_a>; //第0个状态对应于"default"状态, 对应的引脚在pinctrl-0里
    pinctrl-1 = <&state_1_node_a>; //第1个状态对应于"sleep"状态, 对应的引脚在pinctrl-1里面
};

//service
pincontrol {
    state_0_node_a {
```

联合会员

找找看

常用链接

- 我的随笔
- 我的评论
- 我的参与
- 最新评论
- 我的标签

我的标签

- time(1)
- scanf()(1)
- Gtk(1)
- gets()(1)
- fgets()(1)
- CRC校验(1)

随笔分类

- Android (75)
- AndroidTV(1)
- Android上层(14)
- Android手机相关(11)
- Binder(10)
- C++相关(38)
- C语言相关(14)
- eBPF(1)
- git操作(12)
- GStreamer(1)
- Java(24)
- Linux shell(4)
- Linux电源管理和功耗(19)
- linux命令(10)
- Linux驱动相关(84)

```
function = "uart0";
groups = "u0rxtx", "u0rtscts";
};

state_1_node_a {
    function = "gpio";
    groups = "u0rxtx", "u0rtscts";
};
};
```



上面的是对一组引脚的复用，在不同状态下复用为uart引脚或gpio引脚，称为“Generic pin multiplexing node” (复用节点)。还有一种配置叫做“Generic pin configuration node” (配置节点)是对引脚功能的配置，不同的状态(default、idle、sleep...)配置成不同的功能。

一个配置节点的例子如下：



```
//client节点
device {
    pinctrl-names = "default", "sleep"; //使用pinctrl-names来表示设备的状态(state),这里1
    pinctrl-0 = <&state_0_node_a>; //第0个状态对应于"default"状态, 对应的引脚在pinctrl-0里
    pinctrl-0 = <&state_1_node_a>; //第1个状态对应于"sleep"状态, 对应的引脚在pinctrl-1里面
};

//service节点, controller来提供服务
pincontroller {
    state_0_node_a { //复用节点
        function = "uart0";
        groups = "u0rxtx", "u0rtscts";
    };
    state_1_node_a { //复用节点
```

联合会员

更多

随笔档案

- 2025年5月(2)
- 2025年4月(6)
- 2025年3月(8)
- 2025年2月(14)
- 2025年1月(3)
- 2024年12月(2)
- 2024年11月(5)
- 2024年10月(1)
- 2024年9月(4)
- 2024年8月(11)
- 2024年7月(14)
- 2024年6月(6)
- 2024年5月(3)
- 2024年4月(5)
- 2024年3月(6)

更多

文章分类

Linux驱动(1)

阅读排行榜

- 1. Android.bp学习笔记(31362)
- 2. Patch打补丁学习笔记(25047)
- 3. fstab文件详解(19998)
- 4. 世界各个地区WIFI 2.4G及5G信道划分表（附无线通信频率分配表）(18389)
- 5. SDIO接口(17714)

评论排行榜

- 1. Linux内核红黑树2—移植笔记(2)

```
groups = "u0rxtx", "u0rtscts";
output-high; //输出高电平

};

};
```

不论是复用节点还是配置节点，都是来操作这些引脚。对于一个client它可以指定多个状态(state)，在每一个状态下都可以指定对应的子节点来描述它的状态。子节点在controller服务侧实现，子节点可以是一个复用节点(把对应的引脚复用成某个功能)也可以是一个配置节点(把对应的引脚配置成某个状态)。

对于client节点，其设备树书写格式基本上是一致的，统一的。但是对于服务侧的controller节点的写法就五花八门了，有些根本就没有function和group，可以说的上是毫无格式。

举几个实际使用的例子：

1.imx6ull的

```
//client端：
@uart1 {
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_uart1>;
    status = "okay";
};

//pincontroller服务端
pinctrl_uart1: uart1grp {
    fsl.pins = <MX6UL_PAD_UART1_TX_DATA__UART1_DCE_TX, //名字为UART1_TX的引脚被复用为UART1_DCE_TX, //这里写的是两个宏，所以没有加"&"
    MX6UL_PAD_UART1_RX_DATA__UART1_DCE_RX>;
};
```

联合会员

- 2. 内存管理-40-watermark内存水位(1)
- 3. Android对重启原因的记录(1)
- 4. MTK Powerhal 学习笔记(1)
- 5. Linux内核机制—mutex(1)

推荐排行榜

- 1. Patch打补丁学习笔记(4)
- 2. dm-verity简介 —— （1）(3)
- 3. Android音频(9)—— 音量调节(2)
- 4. 世界各个地区WIFI 2.4G及5G信道划分表（附无线通信频率分配表）(2)
- 5. fstab文件详解(2)

最新评论

1. Re:内存管理-40-watermark内存水位
博主您好，请问文中提及到的《内存管理-39-lowmem_reserve低端预留内存》有文章链接吗

--凛冬将至*

2. Re:世界各个地区WIFI 2.4G及5G信道划分表（附无线通信频率分配表）
图片都失效了

--orz_2333

3. Re:Android对重启原因的记录
你好请问文中说的 prop.txt 在什么位置

--CCint3

4. Re:Linux regulator系统 （1）
常供电的设备树节点该怎么写呢？

--潇汀

5. Re:iowait学习笔记
在实际上IO服务时间非常长，但由于系统中同时存在CPU密集型任务掩盖了iowait这个通过什么能看出来？？



2.rk3288平台的



```
//client端

@uart0 {
    pinctrl-names = "default";
    pinctrl-0 = <uart0_xfer &uart0_cts &uart0_rts>; //它使用三个节点来表示三组引脚。
    status = "okay";
};

//pincontroller服务端

gpio4_uart0 {
    uart0_xfer: uart0-xfer {
        rockchip,pins = <UART0BT_SIN>, <UART0BT_SOUT>; //使用rockchip,pins来指定使用哪
        rockchip,pull = <VALUE_PULL_DISABLE>; //这两个字段来配置这些引脚的参数
        rockchip,drive = <VALUE_DRV_DEFAULT>;
    };
    uart0_cts: uart0-cts {
        rockchip,pins = <UART0BT_CTSN>; //这里写的是两个宏，所以没有加"&", 这里其实是指定了i
        rockchip,pull = <VALUE_PULL_DISABLE>;
        rockchip,drive = <VALUE_DRV_DEFAULT>;
    };
    uart0_rts: uart0-rts {
        rockchip,pins = <UART0BT_RTSN>;
        rockchip,pull = <VALUE_PULL_DISABLE>;
        rockchip,drive = <VALUE_DRV_DEFAULT>;
    };
    uart0_
    联合会员
    rockchip,pins = <FUNC_IO_GPIO(UART0BT_RTSN)>;
```

--consolexinhun

```
        rockchip,drive = <VALUE_DRV_DEFAULT>;
    };
};
```

虽然pincontroller服务端没有统一的格式，但是其里面的概念还是一样的，使用到哪些引脚，这些引脚会被归为一组一组，这些引脚会被复用为某一个功能。

3. 一个设备使用多个gpio时的设备树的设置方法

```
pcie0: qcom,pcie@xxx {
    compatible = "qcom,pci-msm";
    pinctrl-names = "default", "sleep";
    pinctrl-0 = <&pcie0_clkreq_default &pcie0_perst_default &pcie0_wake_default>; //
    pinctrl-1 = <&pcie0_clkreq_sleep &pcie0_perst_default &pcie0_wake_default>; //gr
}
```

4. 另外还有一种情况，对于一组gpio，在不同state下pin_func定义不同的情况，如wifi，在active状态设置为wifi功能，在suspend状态下设置为普通gpio。

```
pinctrl@fd511000{
    ...
    pmx-wcnss-5wire-active{
        qcom,pins= <&gp 40>, <&gp 41>, <&gp 42>, <&gp43>, <&gp44>;
        qcom,pin-func= <1>;
        qcom,r
        label= 1v /使用label去指定function
        wcnss-5wire-active:wcns-active {
```

联合会员

```
        drive-strength= <6>; / * 6MA */
        bias-pull-up;
    };
};

pmx-wcnss-5wire-suspend{
    qcom,pins= <&gp 40>, <&gp 41>, <&gp 42>, <&gp43>, <&gp44>;
    qcom,pin-func= <0>;
    qcom,num-grp-pins= <5>;
    label= "wcnss-5wire-suspend"; //使用label去指定function
    wcnss-5wire-sleep:wcnss-sleep {
        drive-strength= <6>; / * 6MA */
        bias-pull-down;
    };
};
};
```



二、驱动中怎样使用pinctrl子系统

对于驱动代码中怎样使用pinctrl子系统，这对驱动工程师来说是透明的，我们驱动中基本不用管，当设备切换状态时(对应设备树中pinctrl-names的状态)，对应的pinctrl就会被调用。

1. 在驱动probe之前就先获取了pinctl的各种state



```
__device_attach
bus_for_each_child(&pdev->dev, &data, __device_attach_driver);

联合会员

driver_match_device(drv, dev); //只有驱动和设备树节点匹配上了才会继续往下走
```

```
driver_probe_device(struct device_driver *drv, struct device *dev)
{
    really_probe
    pinctrl_bind_pins //执行到这里了，一定是驱动和设备匹配上后的。
    drv->probe(dev) //然后再probe我们的驱动
}
```



联合会员



```
int pinctrl_bind_pins(struct device *dev)
{
    /*1.先get*/
    dev->pins->p = devm_pinctrl_get(dev);

    /*2.获取各种state下的pin*/
    dev->pins->default_state = pinctrl_lookup_state(dev->pins->p, PINCTRL_STATE_DEFAULT);
    dev->pins->init_state = pinctrl_lookup_state(dev->pins->p, PINCTRL_STATE_INIT);
    /*3.选中一个state进行设置*/
    if (IS_ERR(dev->pins->init_state)) {
        /*这里对client的gpio设备树节点进行解析，并对硬件进行了设置*/
        ret = pinctrl_select_state(dev->pins->p, dev->pins->default_state); /*如果有i
    } else {
        ret = pinctrl_select_state(dev->pins->p, dev->pins->init_state);
    }

#ifdef CONFIG_PM //R上也定义了这个值
    dev->pins->sleep_state = pinctrl_lookup_state(dev->pins->p, PINCTRL_STATE_SLEEP);
    dev->pins->idle_state = pinctrl_lookup_state(dev->pins->p, PINCTRL_STATE_IDLE);
#endif

    return 0;
}
```



2. 若是非要自己调用，也有函数

```
struct pin      > pinctrl_get_select_default(struct device *dev) //
struct pin      > pinctrl_get_select(struct device *dev, const char
void devm_pinctrl_put(struct pinctrl *p); //释放pinctrl配置资源，通常不需要我们调用。
```

联合会员

3. pinctrl使用步骤

/*1.驱动获取对应的pinctrl节点*/

struct pinctrl *devm_pinctrl_get(struct device *dev)



```
devm_pinctrl_get
    pinctrl_get(dev);
    find_pinctrl(dev); //从pinctrl_list中找到属于此dev的struct pinctrl
    create_pinctrl(dev, NULL);
    pinctrl_dt_to_map(p, pctldev); //解析设备端(client)设备树种的配置, ,
    of_find_node_by_phandle(phandle); //通过phandle找到config节点
    dt_to_map_one_config(p, pctldev, statename, np_config); //解析设备树的
```



/*2.获取各种state的gpio配置*/

struct pinctrl_state *pinctrl_lookup_state(struct pinctrl *p, const char *name) //name =

"xxx_active"/"xxx_sleep"/"xxx_default"

```
struct pinctrl_state *pinctrl_lookup_state(struct pinctrl *p, const char *name)
    find_state(p, name); //通过名字返回对应的pinctrl_state
```

/*3.将上面获取的指定state状态设置到硬件中*/

int pinctrl_select_state(struct pinctrl *p, struct pinctrl_state *state)



```
int pinctrl_select_state(struct pinctrl *p, struct pinctrl_state *state)
    pinctrl_commit_state(p, state);
    pi    联合会员    'r    ing);
    p-    专    设置为null
    pinmux_enable_setting(setting); //对于每一个setting都调用这个函数
```

```
struct pinctrl_ops *pctlops = pctldev->desc->pctlops;
pctlops->get_group_pins(pctldev, setting->data.mux.group, &pins, &num_pins);
pin_request(pctldev, pins[i], setting->dev_name, NULL); //对每一个引脚进行配置
ops->set_mux(pctldev, setting->data.mux.func, setting->data.mux.group);
```



三、对pmic的gpio的获取和使用

上面描述的都是对Soc的gpio的配置，pmic的gpio的配置和使用和之前对gpio的使用方法基本一致。

1. 相关函数：



```
int of_get_named_gpio(struct device_node *np, const char *propname, int index)
int gpio_request(unsigned gpio, const char *label)
int gpio_direction_input(unsigned gpio)
int gpio_direction_output(unsigned gpio, int value)
int gpio_get_value(unsigned int gpio)
void gpio_set_value(unsigned int gpio, int value)
```



2. 举例

设备树节点中gpio配置

```
qcom,otg_en-gpio = <&pm8150l_gpios 10 0x00>;
```

驱动中gpio获取：

```
of_get_named_gpio(np, "qcom,otg_en-gpio", 0);
//或
```

联合会员

```
gpio_request(en_gpio, "qcom,otg_en-gpio");
```

四、总结

1. pinctrl节点，也即是service的节点的驱动中是没有解析设备树的，在设备驱动，也即client的驱动中设置的时候才会去解析设备树，进行实际的硬件配置。若是只是service中配置了，但是没有client使用，没有什么影响。
2. 一般是驱动自身，在suspend回调中设置为sleep state，resume时设置为avtive state。平台设备驱动在probe()时会自动设置为default state，其它驱动在probe()内可以进行设置。

参考：

pinctrl基础简介：<http://www.voidcn.com/article/p-vowhhncl-xp.html>

gpio pinctrl的使用demo：https://dongka.github.io/2018/02/25/pinctrl/sunxi_pinctrl/

分类: [Linux驱动相关](#)

好文要顶

关注我

收藏该文

微信分享



Hello-World3
粉丝 - 11

联合会员

0

0

[+加关注](#)